

Capgemini - 100 Technical Interview Questions

Capgemini

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. What is constructor overloading? Can constructors be inherited?

Answer:

Constructor overloading means a class has multiple constructors with different parameter lists, letting objects be created in different ways (e.g. `Point()` vs `Point(x,y)`). Constructors are NOT inherited by subclasses - a subclass must define its own constructors, though it can call a parent constructor explicitly via `super()`.

Q2. What is the difference between method overloading and method overriding? Give a code example of each.

Answer:

Overloading means multiple methods with the same name but different parameter lists in the same class (resolved at compile time), e.g. `add(int,int)` and `add(double,double)`. Overriding means a subclass provides its own implementation of a method already defined in its parent with the same signature (resolved at runtime), e.g. `Dog` overriding `Animal's makeSound()`.

Q3. What happens when a class implements an interface but does not implement all its methods?

Answer:

If a class implements an interface but doesn't implement all its abstract methods, the class itself must be declared abstract; otherwise, it is a compile-time error since an interface is a contract that guarantees every method has an implementation somewhere in the concrete class hierarchy.

Q4. Explain the four pillars of Object-Oriented Programming with a real-world example for each.

Answer:

The four pillars are: **Encapsulation** - bundling data and methods together and hiding internal state (e.g. a bank account class exposing `deposit()/withdraw()` but hiding the raw balance field); **Abstraction** - exposing only relevant details, like a car's steering wheel hiding the mechanics underneath; **Inheritance** - a Manager class reusing and extending an Employee class; **Polymorphism** - a `draw()` method behaving differently for a Circle vs a Square object called through a common Shape reference.

Four Pillars of OOP

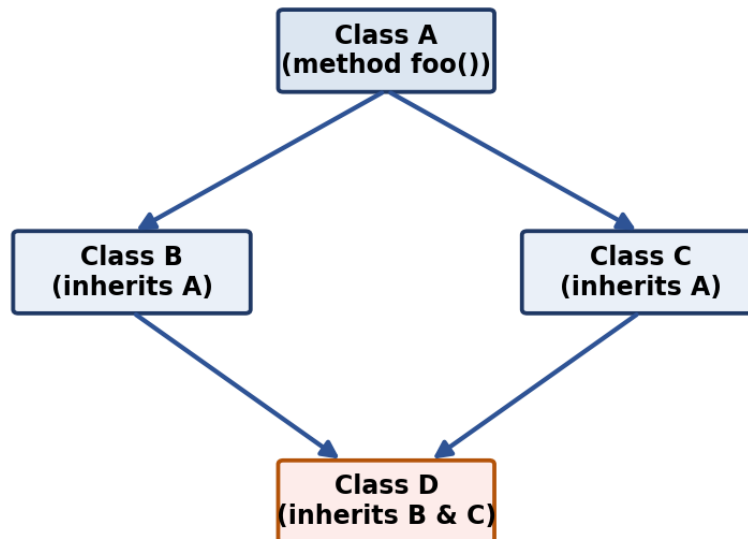


Q5. What is the 'diamond problem' in multiple inheritance, and how does Java avoid it?

Answer:

The diamond problem arises when a class inherits from two classes that both inherit from a common base class, creating ambiguity about which inherited method version to use. Java avoids it for classes by disallowing multiple inheritance of classes entirely (a class can extend only one class), and for interfaces (Java 8+ default methods) it forces the implementing class to explicitly override the conflicting method if two interfaces provide the same default.

Diamond Problem: which foo() does D inherit - via B or via C?



Q6. Explain the Open/Closed Principle with a practical example.

Answer:

The Open/Closed Principle says classes should be open for extension but closed for modification. Practical example: instead of adding an if-else chain to a PaymentProcessor class every time a new payment method is added, define a PaymentMethod interface and add new payment types as new classes implementing it - existing code doesn't need to change.

Q7. What is method hiding, and how is it different from method overriding?

Answer:

Method hiding occurs when a subclass defines a static method with the same signature as a static method in its parent - the parent's version isn't overridden, just hidden, and which one runs depends on the reference type used to call it (resolved at compile time). Method overriding, in contrast, applies to instance methods and is resolved at runtime based on the actual object type.

Q8. Explain the concept of late binding vs early binding with an example.

Answer:

Early (static) binding resolves which method to call at compile time, based on the reference type - used for overloaded and static methods. Late (dynamic) binding resolves the method call at runtime, based on the actual object type - used for overridden instance methods, which is the basis of runtime polymorphism.

Q9. What design pattern would you use to ensure only a single instance of a class exists, and how would you implement it thread-safely?

Answer:

The Singleton design pattern ensures only one instance of a class exists. A thread-safe implementation typically uses either double-checked locking with a volatile instance field, an eagerly-initialized static final instance, or (in Java) a single-element enum, all of which avoid two threads creating two separate instances under concurrent access.

Q10. What is polymorphism? Differentiate between compile-time and runtime polymorphism with examples.

Answer:

Polymorphism lets the same interface behave differently depending on the actual object. Compile-time (static) polymorphism is resolved during compilation - method/operator overloading is the classic example. Runtime (dynamic) polymorphism is resolved during execution based on the actual object type - method overriding via a base-class reference is the classic example.

Q11. What is an object in OOP, and how is it different from a class?

Answer:

An object is a concrete instance of a class, with its own state (field values) stored in memory, while a class is the blueprint/template that defines what fields and methods its objects will have. 'Car' is the class; 'myRedCar' is an object of that class.

Q12. What is a functional interface in Java, and how does it relate to lambda expressions?

Answer:

A functional interface is an interface with exactly one abstract method (it may have default/static methods too), e.g. Runnable or Comparator. Java's lambda expressions are essentially shorthand implementations of a functional interface's single abstract method, letting you write (a, b) -> a.compareTo(b) instead of an anonymous inner class.

Q13. Explain the purpose of the 'this' and 'super' keywords in Java.

Answer:

'this' refers to the current object instance, used to distinguish instance fields from parameters with the same name or to pass the current object elsewhere. 'super' refers to the immediate parent class, used to call a parent's constructor, method, or access a parent field that's been hidden by the subclass.

Q14. Explain encapsulation and why it is important for maintainable software.

Answer:

Encapsulation hides an object's internal state behind private fields and exposes controlled access through public getters/setters or methods. It matters because it lets you change internal implementation without breaking external code, enforce validation (e.g. reject a negative balance), and reduces unintended coupling between modules.

Q15. Describe a scenario from your own project where you used inheritance, and explain why it was the right choice.

Answer:

Example answer structure: describe a project where a base class captured shared behaviour (e.g. a base Notification class with a send() method) and subclasses (EmailNotification, SMSNotification) overrode send() with channel-specific logic - explain why this avoided duplicating shared validation/logging code and made adding a new channel a matter of adding one new subclass.

Q16. Explain the difference between an abstract class and an interface. When would you choose one over the other?

Answer:

An abstract class can mix abstract and concrete methods, hold state (fields), and a class can extend only one abstract class. An interface (pre-Java 8) only declares method signatures with no implementation, and a class can implement many interfaces. Use an abstract class when subclasses share common state/behaviour; use an interface when you only need to guarantee a contract across unrelated classes.

DBMS & SQL

Q17. Write a SQL query to find the Nth highest salary using a window function like RANK() or DENSE_RANK().

Answer:

SELECT name, salary FROM (SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM Employee) ranked WHERE rnk = N (substituting the desired N). DENSE_RANK() assigns ranks without gaps even when there are salary ties, making it more robust than ROW_NUMBER() for this use case.

Q18. How would you optimize a slow-running SQL query? Walk through your debugging approach.

Answer:

Approach: first run EXPLAIN/EXPLAIN ANALYZE to see the query plan and spot full table scans; check whether the filtered/joined columns have appropriate indexes; look for functions applied to indexed columns (which can prevent index use); check for unnecessary SELECT * pulling extra columns; consider rewriting correlated subqueries as joins; and check table statistics are up to date so the optimizer picks a good plan.

Q19. Explain optimistic vs pessimistic locking in database transactions.

Answer:

Pessimistic locking acquires a lock on a row before reading/modifying it, blocking other transactions until it's released - safe under high contention but can hurt throughput. Optimistic locking assumes conflicts are rare: it reads data without locking, then checks (e.g. via a version number) at commit time whether the data changed since it was read, retrying if a conflict is detected - better throughput when conflicts are infrequent.

Q20. What is a stored procedure, and what are its advantages over writing ad-hoc queries?

Answer:

A stored procedure is a precompiled, named collection of SQL statements stored in the database and invoked as a single call. Advantages include reduced network round-trips (one call instead of many statements), better security (grant execute rights without exposing underlying tables directly), reusability across applications, and a cached execution plan that can improve performance over repeatedly sent ad-hoc queries.

Q21. Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

SELECT e.name, e.salary, e.department FROM Employee e WHERE e.salary > (SELECT AVG(salary) FROM Employee e2 WHERE e2.department = e.department). The correlated subquery recomputes the department's average salary for each row and compares it to that employee's own salary.

Q22. What is the difference between OLTP and OLAP systems?

Answer:

OLTP (Online Transaction Processing) systems handle many short, frequent read/write transactions (e.g. an ATM system) and are optimized for fast inserts/updates on normalized schemas. OLAP (Online Analytical Processing) systems handle complex, read-heavy analytical queries over large historical datasets (e.g. a sales dashboard) and are optimized with denormalized/star schemas for fast aggregation.

Q23. Explain database isolation levels and the anomalies (dirty read, non-repeatable read, phantom read) each one prevents.

Answer:

Isolation levels control how much one transaction can see of another's uncommitted work: Read Uncommitted allows dirty reads (seeing uncommitted changes); Read Committed prevents dirty reads but allows non-repeatable reads (a value changes between two reads in the same transaction); Repeatable Read prevents that but allows phantom reads (new rows appearing in a repeated range query); Serializable prevents all three by fully isolating transactions, at the cost of concurrency.

Q24. Explain the difference between a correlated subquery and a regular subquery.

Answer:

A regular (uncorrelated) subquery executes once, independently of the outer query, and its result is used by the outer query. A correlated subquery references a column from the outer query and is conceptually re-evaluated for each row processed by the outer query, e.g. finding employees who earn more than the average salary of their own department.

Q25. Explain referential integrity and what happens when you try to delete a row referenced by a foreign key.

Answer:

Referential integrity ensures a foreign key value always points to an existing row in the referenced table (or is NULL). Trying to delete a row that is still referenced by a foreign key will fail with a constraint violation error, unless the foreign key was defined with ON DELETE CASCADE (deletes dependent rows too) or ON DELETE SET NULL (nulls out the reference) behavior.

Q26. What is a trigger, and give an example of when you would use one.

Answer:

A trigger is a stored procedure automatically executed in response to an INSERT, UPDATE, or DELETE event on a table. Example: a trigger that automatically logs the old and new values of a Salary column into an AuditLog table whenever an Employee row is updated, without requiring the application code to do it explicitly.

Q27. How does a B-Tree index improve query performance internally?

Answer:

A B-Tree index keeps keys sorted in a balanced tree structure with high fan-out, so the database can locate a row's location in only a handful of disk-page reads (typically $O(\log n)$) instead of scanning every row, by repeatedly narrowing the search range as it walks down the tree from the root to a leaf pointing at the actual row.

Q28. What are ACID vs BASE properties, and where would you prefer a BASE-compliant (NoSQL) system?

Answer:

ACID (Atomicity, Consistency, Isolation, Durability) prioritizes strict correctness and is the model used by traditional relational databases. BASE (Basically Available, Soft state, Eventual consistency) relaxes strict consistency in favor of availability and partition tolerance, common in NoSQL systems. You'd prefer a BASE system for massive-scale, highly available workloads (e.g. a social media feed) where brief inconsistency is acceptable in exchange for uptime and horizontal scalability.

Q29. Explain how you would design a database schema for a simple e-commerce order system (tables and relationships).

Answer:

A minimal e-commerce schema would include: Customers (customer_id PK, name, email), Products (product_id PK, name, price), Orders (order_id PK, customer_id FK, order_date, status), and OrderItems (order_item_id PK, order_id FK, product_id FK, quantity, price_at_purchase) as a junction table capturing the many-to-many relationship between Orders and Products.

Q30. What is database replication, and what is the difference between synchronous and asynchronous replication?

Answer:

Database replication maintains copies of data across multiple servers for redundancy and read scaling. Synchronous replication waits for the replica to confirm the write before the transaction commits (stronger consistency, higher write latency); asynchronous replication commits on the primary immediately and propagates changes to replicas afterward (lower latency, but a small risk of data loss if the primary fails before replicating).

Q31. What is indexing, and what are the trade-offs of adding too many indexes to a table?

Answer:

An index is a data structure (commonly a B-Tree) that lets the database find rows matching a condition without scanning the whole table, dramatically speeding up SELECT queries. The trade-off is that every INSERT/UPDATE/DELETE must also update all indexes on that table, slowing down writes and consuming extra storage - so indexes should be added selectively on columns that are frequently filtered or joined on.

Q32. What is a view in SQL, and how is it different from a materialized view?

Answer:

A view is a virtual table defined by a stored SELECT query - it has no data of its own and is recomputed each time it's queried. A materialized view physically stores the result set on disk and must be refreshed periodically (or on trigger), trading storage and staleness for much faster read access compared to recomputing a complex query every time.

Q33. What is sharding, and how does it help scale a database horizontally?

Answer:

Sharding splits a large table's rows horizontally across multiple database servers (shards), typically based on a shard key (e.g. customer_id range or hash), so no single machine needs to store or query the entire dataset. It improves write and storage scalability but introduces challenges like cross-shard joins/transactions and rebalancing when adding new shards.

Q34. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age >= 18), independent of any other table.

Operating Systems

Q35. Explain the concept of multithreading and its benefits and challenges.

Answer:

Multithreading lets multiple threads within one process run concurrently, sharing memory and resources. Benefits include better CPU utilization on multi-core systems, faster context switching between threads than processes, and easier data sharing. Challenges include race conditions, deadlocks, and the added complexity of correctly synchronizing access to shared data.

Q36. What is spooling, and how does it help manage I/O devices like printers?

Answer:

Spooling (Simultaneous Peripheral Operations On-Line) buffers data intended for a slow I/O device (like a printer) on disk, letting a process 'print' instantly to the spool without waiting for the physical device, while a separate spooler process feeds jobs to the device at its own pace - allowing multiple processes to share one device without conflict.

Q37. What is the difference between a hard link and a symbolic link in a file system?

Answer:

A hard link is essentially another directory entry pointing directly to the same inode (and underlying data) as the original file - deleting the original name still leaves the data accessible via the hard link, and it cannot span file systems. A symbolic (soft) link is a small separate file that stores the path to the target file - it can point across file systems or to directories, but becomes a 'dangling link' if the target is deleted or moved.

Q38. Explain how the Least Recently Used (LRU) page replacement algorithm works, and how you might implement it efficiently.

Answer:

LRU (Least Recently Used) evicts the page that hasn't been used for the longest time when a new page must be loaded and memory is full, based on the assumption that recently used pages are likely to be used again soon. An efficient implementation uses a combination of a hashmap (for $O(1)$ lookup of a page's location) and a doubly linked list ordered by recency, so both finding a page and moving it to the 'most recently used' end take $O(1)$ time.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q39. What is the Banker's Algorithm, and how does it help avoid deadlocks?

Answer:

The Banker's Algorithm avoids deadlock by only granting a resource request if the resulting state is 'safe' - meaning there exists some order in which all processes could still finish given the maximum resources they might request. It simulates allocation before actually granting it, refusing requests that could lead to an unsafe (potentially deadlocked) state.

Q40. Explain what an inode is and what information it stores.

Answer:

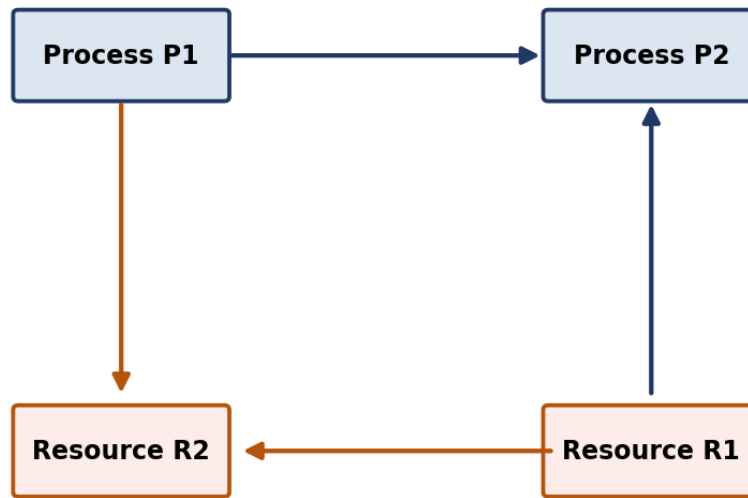
An inode is a data structure in Unix-like file systems that stores a file's metadata: permissions, owner, size, timestamps, and pointers to the disk blocks holding the actual file data - notably, it does not store the file's name (that mapping lives in the containing directory), which is why hard links to the same inode can have different names.

Q41. Explain the four necessary conditions for a deadlock to occur.

Answer:

The four necessary conditions are: **Mutual Exclusion** - at least one resource is held in a non-shareable mode; **Hold and Wait** - a process holds one resource while waiting for another; **No Preemption** - resources can't be forcibly taken away, only released voluntarily; **Circular Wait** - a closed chain of processes each waiting for a resource held by the next. All four must hold simultaneously for a deadlock to occur.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q42. What is a race condition, and how can synchronization primitives like semaphores or mutexes prevent it?

Answer:

A race condition occurs when multiple threads/processes access shared data concurrently and the final result depends on the unpredictable timing/order of execution, potentially producing incorrect results. Synchronization primitives like mutexes (mutual exclusion locks) or semaphores prevent this by ensuring only one thread can enter a critical section that modifies shared data at a time.

Q43. What is the difference between multiprogramming, multitasking, and multiprocessing?

Answer:

Multiprogramming keeps multiple programs in memory so the CPU always has something to execute when one process is waiting on I/O, improving utilization on a single CPU. Multitasking extends this idea to give the illusion of several programs running 'simultaneously' via time-slicing, often in an interactive context. Multiprocessing refers to a system with multiple physical CPUs/cores actually executing instructions in parallel at the same instant.

Q44. Describe a real-world analogy to explain deadlock to a non-technical person.

Answer:

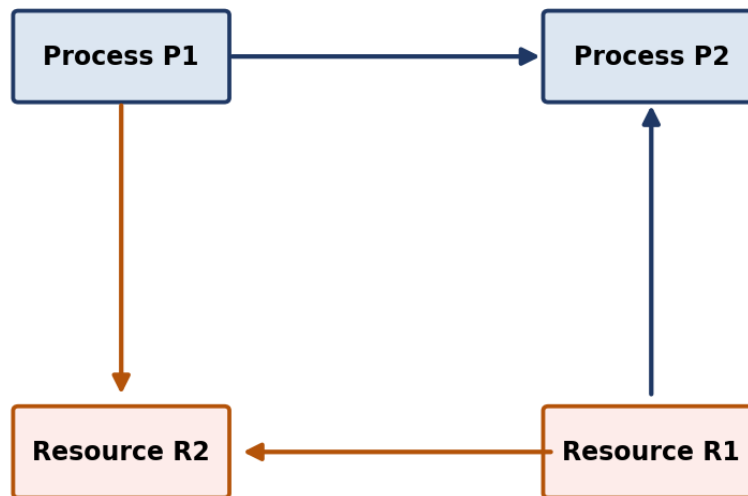
Analogy: imagine four cars at a four-way intersection, each waiting for the car to its right to move first before it can proceed - if all four wait simultaneously, none will ever move, even though each car is only blocked because of the others' identical behavior. That's a deadlock: everyone is waiting for someone else to act first, and no one ever will.

Q45. What is a deadlock, and describe three general strategies for handling deadlocks (prevention, avoidance, detection & recovery).

Answer:

A deadlock is a state where a set of processes are permanently blocked, each waiting for a resource held by another in the set. Prevention negates one of the four necessary conditions upfront (e.g. requiring resources to be requested all at once). Avoidance checks each request dynamically for safety (e.g. Banker's Algorithm). Detection & Recovery lets deadlocks happen but periodically checks for cycles in the resource-allocation graph and recovers by killing/rolling back a process to break the cycle.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)

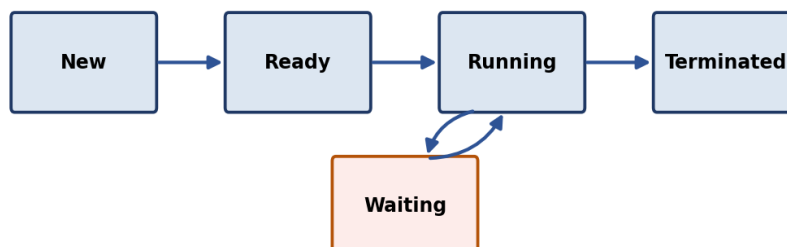


Q46. What is the difference between a process and a thread?

Answer:

A process is an independent program in execution with its own memory space (code, data, heap, stack) and OS-allocated resources. A thread is a lightweight unit of execution within a process that shares the process's memory and resources with other threads of the same process, making thread creation and context switching cheaper than process creation.

Process State Lifecycle



Q47. Explain the difference between paging and segmentation as memory management techniques.

Answer:

Paging divides both physical and logical memory into fixed-size blocks (pages/frames), eliminating external fragmentation but potentially causing internal fragmentation within a page. Segmentation divides a program into variable-sized logical units (code, stack, heap segments) that map more naturally to how programmers think about memory, but can suffer external fragmentation since segments vary in size.



Q48. How does an operating system ensure fair CPU allocation among multiple processes?

Answer:

The OS ensures fair CPU allocation through its scheduler, using algorithms like Round Robin (equal time slices for each process) or priority-based scheduling with aging (gradually increasing the priority of processes that have waited a long time, to prevent starvation), combined with preemption so no single process can monopolize the CPU indefinitely.

Computer Networks

Q49. What happens during TCP connection termination (four-way handshake)?

Answer:

TCP connection termination typically uses a four-way handshake: the initiating side sends a FIN (finish); the other side ACKs it (it may still be sending data); once the other side is also done sending, it sends its own FIN; the initiating side ACKs that FIN, formally closing the connection. Two separate FIN/ACK exchanges are needed because TCP connections are full-duplex - each direction must be closed independently.

Q50. Explain the seven layers of the OSI model and the role of each layer.

Answer:

The seven OSI layers, bottom to top: **Physical** - raw bit transmission over a medium; **Data Link** - framing and MAC addressing between directly connected nodes; **Network** - logical addressing and routing between networks (IP); **Transport** - end-to-end reliable/unreliable delivery (TCP/UDP); **Session** - managing sessions/connections between applications; **Presentation** - data formatting, encryption, compression; **Application** - the actual protocols apps use (HTTP, FTP, DNS).

OSI Model - 7 Layers



Q51. What is the difference between TCP and UDP? Give an example use case for each.

Answer:

TCP is connection-oriented, establishes a session via a handshake, guarantees reliable and ordered delivery through acknowledgments and retransmission, but has more overhead - used for web pages, file transfers, and email. UDP is connectionless, sends packets with no delivery guarantee or ordering, but is faster with lower overhead - used for DNS lookups, video calls, and online gaming where speed matters more than occasional lost packets.

TCP	UDP
Connection-oriented Reliable, ordered delivery Slower (handshake+ack) e.g. HTTP, FTP, Email	Connectionless Best-effort, unordered Faster, lower overhead e.g. DNS, video/voice calls

Q52. Describe how you would troubleshoot a situation where a web application is reachable via IP but not via domain name.

Answer:

Troubleshooting approach: first verify DNS resolution actually works (nslookup/dig the domain) since reachability by IP but not domain strongly suggests a DNS issue - check the domain's A/AAAA/CNAME records are correctly configured and propagated, check the local machine's DNS resolver/cache and hosts file, and confirm the domain registrar/DNS provider settings point to the correct IP or load balancer.

Q53. What is NAT, and why is it necessary given the limited IPv4 address space?

Answer:

NAT (Network Address Translation) allows multiple devices on a private network to share a single public IP address when communicating with the internet, translating private IPs to the public IP (and back for return traffic) at the router/gateway. It's necessary because IPv4's roughly 4.3 billion addresses are far fewer than the number of internet-connected devices worldwide.

Q54. Explain the difference between IPv4 and IPv6.

Answer:

IPv4 uses 32-bit addresses (about 4.3 billion possible addresses, written as four decimal octets like 192.168.1.1) and is running out of available addresses. IPv6 uses 128-bit addresses (an astronomically larger space, written in hexadecimal groups like 2001:0db8::1), and also simplifies header processing and has built-in support for features IPv4 needed extensions for, like auto-configuration.

Q55. What is a subnet mask, and how do you calculate the number of usable hosts in a subnet?

Answer:

A subnet mask defines which portion of an IP address is the network part and which is the host part (e.g. 255.255.255.0 means the first 24 bits are network, the last 8 are host). For a /24 subnet, there are $2^8 = 256$ total addresses, minus 2 reserved (network address and broadcast address), giving 254 usable host addresses.

Q56. Explain the difference between a hub, a switch, and a router.

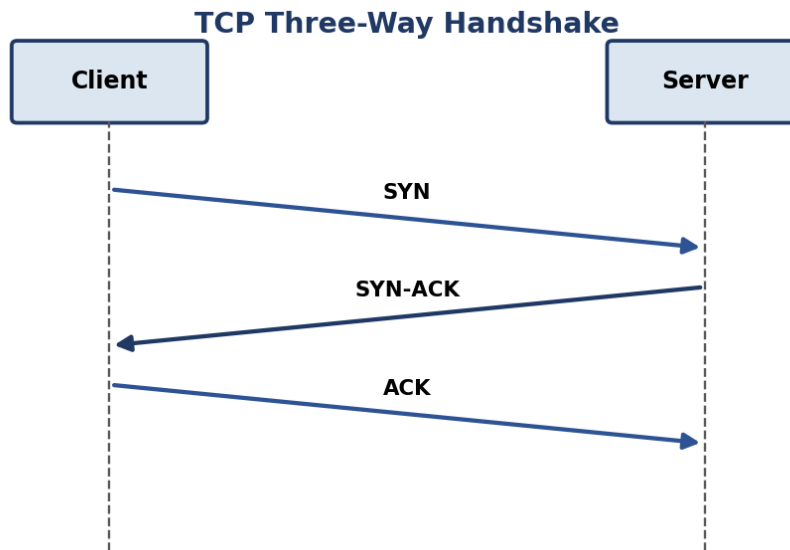
Answer:

A hub is a simple, dumb device that repeats incoming signals to every port (Layer 1), causing collisions and wasted bandwidth. A switch operates at Layer 2, learning MAC addresses and forwarding frames only to the specific port the destination device is on, reducing collisions. A router operates at Layer 3, forwarding packets between different networks based on IP addresses, and is what connects a LAN to the internet.

Q57. Explain the TCP three-way handshake process for establishing a connection.

Answer:

The three-way handshake establishes a reliable TCP connection: the client sends a SYN (synchronize) packet to the server; the server responds with a SYN-ACK, acknowledging the client's SYN and sending its own; the client responds with an ACK, acknowledging the server's SYN. After this, both sides have confirmed each other's readiness to send/receive, and data transfer begins.



Q58. What is the role of DNS, and walk through what happens when you type a URL into a browser.

Answer:

DNS translates a human-readable domain name into an IP address. When you type a URL: the browser checks its cache, then the OS cache, then queries a DNS resolver, which walks the DNS hierarchy (root -> TLD -> authoritative nameserver) if needed to resolve the domain to an IP; the browser then opens a TCP connection (and TLS handshake for HTTPS) to that IP, sends an HTTP request, and renders the response.

Data Structures & Algorithms

Q59. How would you find the kth largest element in an unsorted array efficiently?

Answer:

One efficient approach uses a min-heap of size k: iterate through the array, pushing elements onto the heap and popping the smallest whenever the heap exceeds size k - after processing all elements, the heap's root is the kth largest, achieving $O(n \log k)$ time. Alternatively, Quickselect (a partition-based approach related to Quicksort) finds it in average $O(n)$ time.

Q60. Explain topological sorting and where it is used.

Answer:

Topological sorting produces a linear ordering of a Directed Acyclic Graph's (DAG) vertices such that for every directed edge $u \rightarrow v$, u comes before v in the ordering. It's used wherever tasks have dependencies that must be respected, such as course prerequisite ordering, build systems compiling files in the right order, or task scheduling pipelines.

Q61. Explain amortized time complexity using the example of a dynamic array's append operation.

Answer:

Amortized time complexity averages the cost of an operation over a sequence of operations, even if occasional individual operations are expensive. A dynamic array's append is usually $O(1)$, but when the underlying fixed-size array is full, it must allocate a new (typically double-sized) array and copy all existing elements - an $O(n)$ operation. Because this expensive resize happens increasingly rarely as the array grows, the average cost per append across many operations still works out to $O(1)$ amortized.

Q62. Explain how a trie data structure works and where it is useful (e.g., autocomplete).

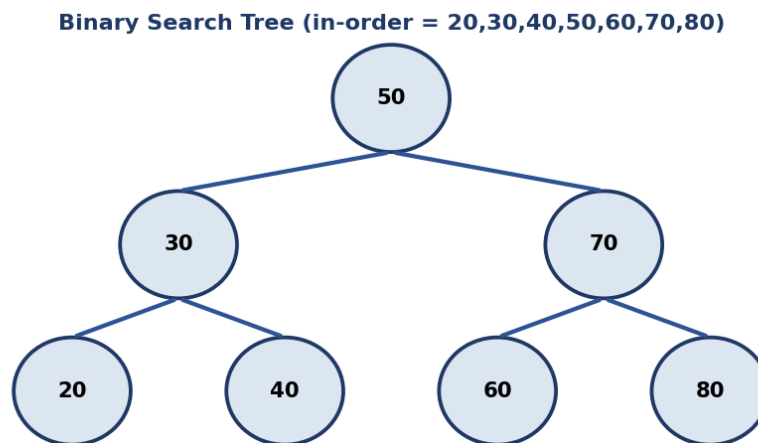
Answer:

A trie (prefix tree) stores strings character by character along tree edges, where each path from the root represents a prefix, and nodes are often marked to indicate a complete word ends there. It's especially useful for prefix-based operations like autocomplete/typeahead suggestions, spell-checking, and IP routing tables, since searching or inserting a string takes time proportional to the string's length rather than the number of stored strings.

Q63. What is the difference between a balanced and an unbalanced binary search tree, and why does balance matter?

Answer:

A balanced BST keeps its height close to $\log n$ by rebalancing after insertions/deletions (e.g. AVL trees, Red-Black trees), guaranteeing $O(\log n)$ search/insert/delete. An unbalanced BST can degenerate into something resembling a linked list if elements are inserted in sorted order without rebalancing, degrading all operations to $O(n)$. Balance matters because it's what actually delivers the logarithmic performance a BST is chosen for in the first place.



Q64. What is memoization, and how does it improve the performance of recursive algorithms?

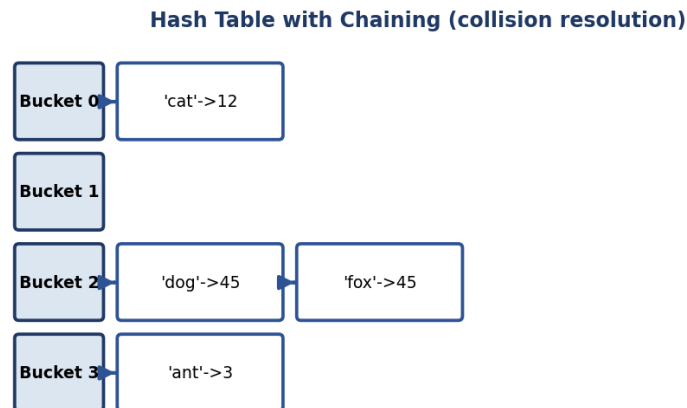
Answer:

Memoization stores the result of a function call keyed by its input parameters (often in a hashmap or array), so if the same inputs occur again, the cached result is returned instantly instead of recomputing it. This turns algorithms with exponential-time naive recursion (like plain recursive Fibonacci) into polynomial-time algorithms by eliminating redundant recursive calls on the same subproblem.

Q65. Explain how a hash table works internally, including collision resolution strategies.

Answer:

A hash table maps keys to array indices using a hash function, giving average $O(1)$ time for insert/search/delete. Since different keys can hash to the same index (a collision), collisions are typically resolved via chaining (each bucket holds a linked list/array of entries that hashed there) or open addressing (probing for the next free slot in the array itself, e.g. linear or quadratic probing).



Q66. Explain the two-pointer technique with an example problem.

Answer:

The two-pointer technique uses two indices moving through a data structure (often from opposite ends or at different speeds) to avoid nested loops. Example: given a sorted array, to find a pair summing to a target, start one pointer at the beginning and one at the end - if the sum is too large, move the right pointer left; if too small, move the left pointer right - solving the problem in $O(n)$ instead of the $O(n^2)$ brute force of checking every pair.

Q67. Explain how binary search works and derive its time complexity.

Answer:

Binary search repeatedly divides a sorted array in half: compare the target to the middle element, and discard the half that can't contain the target, narrowing the search range each time. Since the search space is halved every step, the time complexity is $O(\log n)$ - a huge improvement over linear search's $O(n)$ for large sorted datasets.

Q68. How would you merge two sorted arrays/linked lists efficiently?

Answer:

Use a two-pointer merge: maintain one pointer into each sorted array/list, repeatedly compare the elements they point to, append the smaller one to the result, and advance that pointer - continue until one input is exhausted, then append the remainder of the other. This runs in $O(n + m)$ time, which is also the core merge step used inside Merge Sort.

Q69. Explain the difference between time complexity and space complexity with examples.

Answer:

Time complexity measures how an algorithm's runtime grows as input size increases (e.g. $O(n)$ for a single loop through n elements). Space complexity measures how much additional memory an algorithm needs relative to input size (e.g. $O(1)$ for an in-place swap vs $O(n)$ for creating a copy of the array). Both are usually expressed using Big-O notation and both matter when choosing an algorithm for constrained environments.

Q70. Compare Merge Sort and Quick Sort - discuss best/worst case complexity, stability, and when you'd choose one over the other.

Answer:

Merge Sort has guaranteed $O(n \log n)$ time in best, average, and worst cases, is stable, but requires $O(n)$ extra space for merging. Quick Sort also averages $O(n \log n)$ but can degrade to $O(n^2)$ in the worst case (e.g. already-sorted input with a poor pivot choice), is typically not stable, but sorts in-place with better real-world constant factors and cache performance. Choose Merge Sort when stability or guaranteed worst-case performance matters (e.g. external sorting); choose Quick Sort for general in-memory sorting where average-case speed matters more.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q71. What is Big-O notation, and why do we use asymptotic analysis instead of exact runtimes?

Answer:

Big-O notation describes the upper-bound growth rate of an algorithm's running time or space as input size approaches infinity, ignoring constant factors and lower-order terms. We use asymptotic analysis instead of exact runtimes because exact timings vary by hardware, language, and implementation details, while Big-O gives a hardware-independent way to compare how algorithms scale as input grows large.

Q72. What is backtracking, and how is it different from brute force?

Answer:

Backtracking systematically explores all candidate solutions, but abandons ('backtracks' from) a partial candidate as soon as it determines that candidate cannot possibly lead to a valid solution, rather than exploring it to completion. This pruning makes it far more efficient than pure brute force, which would fully explore every possibility regardless of early signs of failure - classic examples include the N-Queens problem and Sudoku solvers.

Coding Problems

Q73. Write a program to find the missing number in an array containing n distinct numbers from 0 to n.

Answer:

Sum formula approach: the expected sum of numbers 0 to n is $n*(n+1)/2$; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - $O(n)$ time, $O(1)$ space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q74. Write a program to merge overlapping intervals given a list of intervals.

Answer:

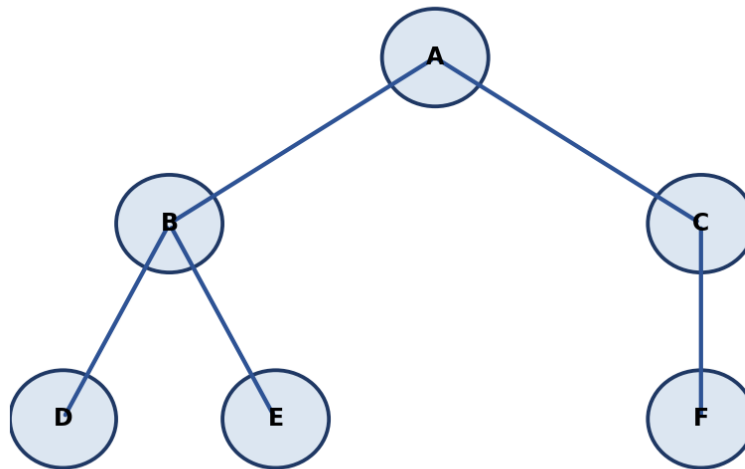
Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in $O(n \log n)$ time due to the initial sort.

Q75. Write a program to count the number of islands in a 2D grid (graph/matrix traversal).

Answer:

Treat the grid as an implicit graph where each land cell ('1') is a node connected to its up/down/left/right land neighbors. Scan every cell; whenever an unvisited land cell is found, increment the island count and run a BFS or DFS from that cell to mark every connected land cell as visited, so it isn't counted again. This runs in $O(\text{rows} * \text{cols})$ time since each cell is visited a constant number of times.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F



Q76. Write a program to implement quicksort or mergesort from scratch.

Answer:

Quicksort: pick a pivot element, partition the array so elements smaller than the pivot come before it and larger elements come after, then recursively sort the two partitions - average $O(n \log n)$, worst case $O(n^2)$ with a poor pivot choice. Mergesort: recursively split the array in half until subarrays of size 1 remain, then repeatedly merge sorted subarrays back together - guaranteed $O(n \log n)$ in all cases, but needs $O(n)$ auxiliary space for merging.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q77. Write a program to check if two strings are anagrams of each other.

Answer:

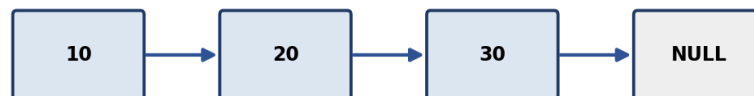
Two common approaches: (1) sort both strings and check if the sorted versions are equal ($O(n \log n)$); or (2) build a frequency count (e.g. a 26-length array for lowercase letters, or a hashmap for general characters) of one string, then decrement counts while scanning the second string - if all counts return to zero and lengths match, they're anagrams ($O(n)$ time, $O(1)$ or $O(k)$ space).

Q78. Write a program to reverse a singly linked list (iterative and recursive approaches).

Answer:

Iterative: maintain a 'previous' pointer (initially null) and a 'current' pointer (initially head); at each step, save current.next, point current.next to previous, then advance previous and current forward - repeat until current is null, then previous is the new head. Recursive: recurse to the end of the list first, then reverse the pointer of each node on the way back up (node.next.next = node; node.next = null). Both run in $O(n)$ time; iterative uses $O(1)$ extra space while recursive uses $O(n)$ call-stack space.

Singly Linked List



Q79. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining n-k elements - this three-step reversal rotates the array by k positions in-place in $O(n)$ time and $O(1)$ extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading $O(n)$ space for simpler code.)

Q80. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On `get(key)`: if present, move that node to the front (most recently used end) and return its value, else return `-1/None`. On `put(key, value)`: if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in $O(1)$.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q81. Write a program to find the middle element of a linked list in a single pass.

Answer:

Use the 'slow and fast pointer' technique: the slow pointer advances one node at a time while the fast pointer advances two nodes at a time - when the fast pointer reaches the end of the list, the slow pointer will be exactly at the middle. This finds the middle node in a single pass, in $O(n)$ time and $O(1)$ space.

Q82. Write a program to check whether a given string is a palindrome.

Answer:

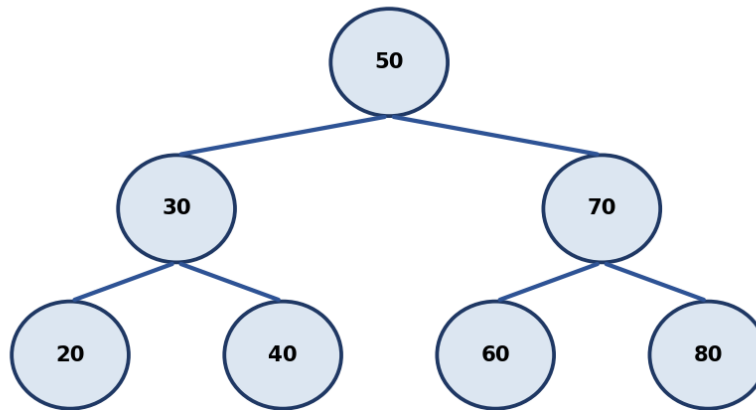
Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in $O(n)$ time and $O(1)$ extra space (ignoring the input itself).

Q83. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in $O(n)$ time.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)

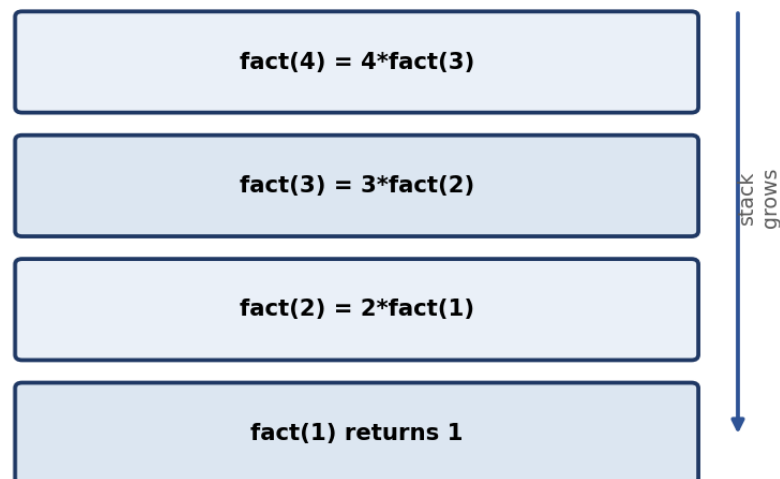


Q84. Write a program to find the factorial of a number using recursion.

Answer:

Recursive definition: $\text{factorial}(n) = n * \text{factorial}(n-1)$, with a base case $\text{factorial}(0) = 1$ (or $\text{factorial}(1) = 1$). Each call multiplies n by the result of the smaller subproblem until the base case is reached, then the multiplications unwind back up the call stack. Time complexity is $O(n)$; space complexity is $O(n)$ due to the recursive call stack.

Call Stack while computing factorial(4)



Q85. Write a program to find the first non-repeating character in a string.

Answer:

Use a hashmap to count the frequency of each character in a single pass through the string, then iterate through the string again in order and return the first character whose count is exactly 1. This runs in $O(n)$ time and $O(k)$ space, where k is the size of the character set.

Q86. Write a program to determine if a given number is prime, and optimize it for large inputs.

Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$), skipping even numbers after checking 2, which reduces the check to roughly $O(\sqrt{n})$. For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in $O(n \log \log n)$, which is far more efficient than testing each number individually.

Programming Language Fundamentals

Q87. Explain Python's Global Interpreter Lock (GIL) and its impact on multithreaded CPU-bound programs.

Answer:

CPython's Global Interpreter Lock ensures only one thread executes Python bytecode at any given instant, even on a multi-core machine, to simplify memory management (reference counting) and avoid certain race conditions internally. This means CPU-bound multithreaded Python code doesn't actually get faster from adding more threads (they contend for the GIL) - for true parallel CPU-bound work, developers typically use multiprocessing (separate processes, each with its own interpreter and GIL) instead.

Q88. What is garbage collection, and how does it help prevent memory leaks?

Answer:

Garbage collection is an automatic memory management process that identifies objects no longer reachable/referenced by the running program and reclaims their memory, so the developer doesn't have to manually free it. This helps prevent memory leaks caused by forgetting to deallocate memory, though a program can still 'leak' memory logically if it keeps unnecessary references alive (e.g. objects sitting in a growing cache that's never cleared).

Q89. What is the difference between '==' and '.equals()' when comparing objects in Java?

Answer:

'==' compares object references - whether two variables point to the exact same object in memory. '.equals()' compares the logical content/value of two objects, based on however the class has overridden it (e.g. String's .equals() compares character sequences). Two distinct String objects with the same text will be '==' false but '.equals()' true, unless string interning makes them the same reference.

Q90. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via malloc/new), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Q91. Explain the difference between pass-by-value and pass-by-reference with an example.

Answer:

Pass-by-value copies the actual value of a variable into the function's parameter, so changes inside the function don't affect the original variable. Pass-by-reference passes a reference to the original variable's memory location, so changes inside the function do affect the original. Example: in a language with true pass-by-reference, `swap(a, b)` can actually swap the caller's variables; with pass-by-value, it would only swap local copies, leaving the caller's variables unchanged.

Q92. What is method chaining, and how would you implement it in a class you design?

Answer:

Method chaining lets you call multiple methods on the same object in a single statement, e.g. `builder.setName('X').setAge(30).build()`. To implement it, each method that would normally return void instead returns 'this' (the current object instance), allowing the next method call to be appended directly onto the previous one's result - commonly used in builder-pattern classes for constructing complex objects readably.

Q93. What is the difference between a compiler and an interpreter?

Answer:

A compiler translates the entire source program into machine code (or another target language) ahead of time, producing an executable that runs independently afterward - errors are caught before execution, and execution tends to be faster since translation work is already done. An interpreter reads and executes source code line-by-line (or statement-by-statement) at runtime without producing a standalone executable, which makes development/debugging faster but execution generally slower since translation happens repeatedly during each run.

Q94. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract Shape class with an abstract `area()` method but a concrete `describe()` method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a Circle class extending Shape and implementing `area()`.

Q95. Explain the difference between checked and unchecked exceptions in Java.

Answer:

Checked exceptions (e.g. `IOException`) are checked by the compiler at compile time - a method must either handle them with try-catch or declare them with 'throws', forcing the caller to acknowledge the possibility of failure. Unchecked exceptions (e.g. `NullPointerException`, extending `RuntimeException`) are not checked at compile time and don't require explicit handling, typically representing programming errors that could occur almost anywhere.

Q96. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like try/catch/finally, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Web & Cloud Fundamentals

Q97. Explain the difference between IaaS, PaaS, and SaaS with real examples of each.

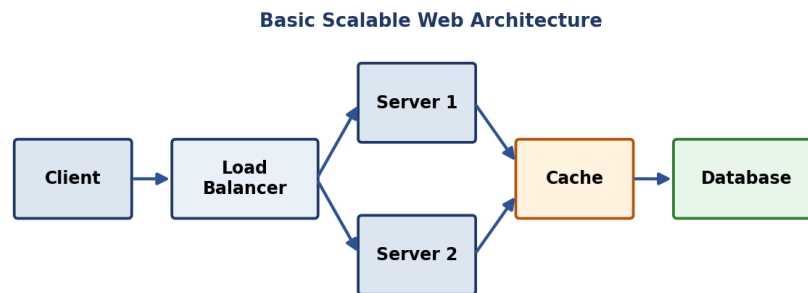
Answer:

IaaS (Infrastructure as a Service) provides raw virtualized compute/storage/networking that you configure yourself, e.g. AWS EC2 or Azure VMs. PaaS (Platform as a Service) provides a managed platform to build and deploy applications without managing the underlying servers, e.g. Heroku or Azure App Service. SaaS (Software as a Service) delivers a complete, ready-to-use application over the internet, e.g. Gmail or Salesforce - you just use it, with no infrastructure or platform management at all.

Q98. What is the purpose of an API Gateway in a microservices-based application?

Answer:

An API Gateway sits in front of a microservices architecture as the single entry point for client requests, routing each request to the appropriate backend service. It commonly also handles cross-cutting concerns centrally - authentication/authorization, rate limiting, request/response transformation, and logging - so individual microservices don't each need to reimplement that logic.



Q99. What is the difference between HTTP and HTTPS, and how is HTTPS implemented under the hood?

Answer:

HTTP transmits data in plaintext, so it's vulnerable to eavesdropping and tampering in transit. HTTPS secures HTTP by layering it on top of TLS/SSL: a handshake negotiates a shared session key and verifies the server's identity via its certificate, and all subsequent HTTP traffic is encrypted using that session key, protecting confidentiality and integrity even over an untrusted network.

Q100. Explain how a CDN improves website performance for geographically distributed users.

Answer:

A CDN caches static content (images, CSS, JS, video) on edge servers distributed across many geographic locations. When a user requests that content, it's served from the nearest edge server instead of traveling all the way to the origin server, reducing round-trip latency, offloading traffic from the origin, and improving load times especially for users far from the origin's data center.

Note: 20 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.